



RELAZIONE PROGETTO SISTEMI OPERATIVI

**SERVIZIO DI CALCOLO DELLA IMPRONTA SHA-256 DI FILE
(proposta 2 con thread e FIFO)**

Mattia Danese (VR502999)

Panoramica

Sviluppare un sistema distribuito (client-server) per il calcolo dell'hash SHA-256 di file, attraverso l'utilizzo di Pthread con monitor e FIFO. SHA ed appartiene ad una famiglia di algoritmi che convertono una stringa di byte di lunghezza arbitraria in una impronta di lunghezza fissata in bit. Nel nostro caso si tratta di una impronta di 256 bit, ovvero 32 byte rappresentabili visivamente come una stringa di 64 simboli esadecimali. Viene utilizzata per sintetizzare contenuti informativi complessi in un formato compatto, è improbabile che due input diversi abbiano la stessa impronta. Usi classici sono per indicizzazione di dati tramite hashing oppure per verifica di integrità, per esempio controllando che l'impronta di un file scaricabile pubblicata ufficialmente combaci con quella computata sulla copia scaricata del file.

Obiettivi

L'obiettivo generale è realizzare un server che permetta multiple computazioni di impronte SHA-256. **Successivamente va realizzato un client** che invia l'informazione di file di input al server e riceva l'impronta risultante appena computata

Architettura del sistema

Il sistema è basato su **un'architettura client-server**, dove uno o più client interagiscono con un singolo server centrale per richiedere il calcolo dell'hash SHA-256 di file.

Client

Ogni istanza del client è un processo separato. Il suo compito è prendere in input un percorso di file, determinare la sua dimensione, generare un nome univoco per una FIFO di risposta temporanea, costruire il messaggio di richiesta secondo un protocollo definito e inviarlo alla FIFO principale del server. Successivamente, attende sulla propria FIFO di risposta il risultato del calcolo SHA-256 o un messaggio di errore dal server

Server

Il server ha il compito principale di **gestire le richieste dei client per il calcolo dell'hash SHA-256 di file, elaborando in modo concorrente** .

Più in dettaglio, il server svolge le seguenti funzioni:

Gestione FIFO Principale: Crea e mantiene aperta una FIFO per ricevere le richieste, usando un descrittore di scrittura aperto per evitare blocchi prematuri.

Ricezione e Parsing Richieste: Legge e analizza i messaggi di richiesta dalla FIFO, estraendo percorso, dimensione del file e percorso della FIFO di risposta del client.

Gestione Thread Pool: Inizializza un numero fisso di thread worker ovvero un "thread pool" nel mio caso di 4 thread all'avvio, pronti ad elaborare nuove richieste.

Accodamento Richieste (Work Queue): Inserisce le richieste in una coda che le **schedula per dimensione del file**, dando priorità ai file più piccoli per migliorare la reattività. L'accesso è sincronizzato con mutex e variabili di condizione.

Elaborazione Richieste (Worker Threads): I worker prelevano richieste dalla coda e:

- **Verificano la Cache:** Controllano se l'hash è già disponibile in memoria.
- **Gestiscono Duplicati:** Se un calcolo è già in corso per lo stesso file, attendono il risultato esistente.
- **Calcolano SHA-256:** Se non in cache e non in corso, calcolano l'hash.
- **Aggiornano Cache:** Aggiungono il risultato alla cache per usi futuri.

Invio Risposte: Spediscono l'hash calcolato (o un errore) alla FIFO di risposta dedicata del client.

Sincronizzazione: Utilizza mutex e variabili di condizione per garantire la coerenza e la thread-safety nell'accesso alle risorse condivise

Meccanismo di comunicazione (FIFO)

La comunicazione tra i processi avviene esclusivamente tramite **FIFO**.

Questo garantisce un canale di comunicazione standard tra client e server.

Modello di concorrenza (Thread pool)

La gestione della concorrenza all'interno del server è realizzata attraverso un **thread pool**. Questo modello permette di gestire ed elaborare più richieste contemporaneamente, ottimizzando l'uso delle risorse di calcolo.

Difficoltà riscontrate e soluzioni adottate

Sincronizzazione Concorrente: Gestire l'accesso sicuro a risorse condivise (coda, cache, elenco richieste pendenti) per evitare race condition e deadlock.

Soluzione: Implementazione di **mutex** e **condition variables** per controllare l'accesso tra i thread.

Gestione Richieste Duplicate: Evitare calcoli ripetuti dello stesso hash per richieste concorrenti sullo stesso file.

Soluzione: Sviluppo del meccanismo delle "richieste pendenti" che permette ai thread di attendere il risultato di un calcolo già in corso.

Parsing e Protocollo: Assicurare la corretta interpretazione dei messaggi di richiesta/risposta tra client e server.

Soluzione: Definizione chiara di un **protocollo** (`protocol.h`) con delimitatori e formati di messaggio standardizzati.

Possibili sviluppi futuri

Ottimizzazione Cache:

Migliorare l'attuale cache (**LISTA CONCATENATA**) con una **cache LRU** per risposte più rapide..

Terminazione Controllata del Server:

Aggiungere un **signal handler** (**SIGINT**, **SIGTERM**) per chiudere le FIFO e liberare tutte le risorse in modo pulito alla chiusura del server.

Gestione Carico Coda:

Implementare un **limite massimo** per la coda di lavoro per prevenire l'eccessivo consumo di memoria sotto carico pesante.

Configurabilità: Rendere configurabili parametri come il **numero di thread worker** e la **dimensione della cache** tramite argomenti da riga di comando.

Timeout Client: Introdurre timeout nelle operazioni di lettura del client per evitare blocchi indefiniti in caso di problemi sul server.

Schema di compilazione ed esecuzione

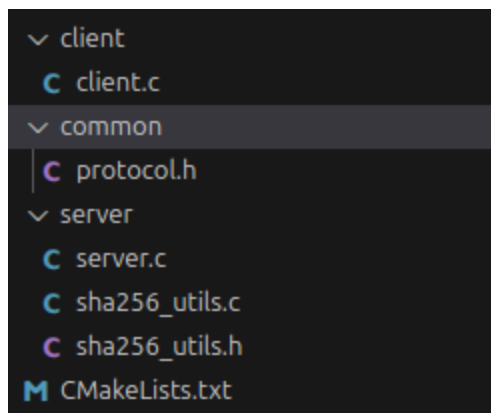
Questa sezione ti guiderà passo passo nella compilazione e nell'esecuzione del sistema client-server, sfruttando **CMake** per compilare il progetto.

Prerequisiti

Assicurati di aver installato sul tuo pc i seguenti strumenti:

- Compilatore **GCC**
- Librerie:
 - **OpenSSL** per le funzioni di calcolo SHA-256.
- Sistema di Build: **CMake**

Struttura del progetto



Compilazione del progetto con CMake (esegui i comandi dove ci sono i punti in un terminale)

1. Crea la directory build e entra in build:
 - `mkdir build`
 - `cd build`
2. Configura il progetto : Esegui CMake dalla directory **build**. Questo comando analizza i file **CMakeLists.txt** e genererà i file di build necessari :
 - `cmake..`
3. Compila il codice :
 - `make`

Al termine della compilazione, troverai gli eseguibili **server** e **client** all'interno della directory **build**.

Esecuzione del sistema (anche qua esegui dove ci sono i punti in un terminale)

Una volta compilati gli eseguibili, puoi avviare il server e i client.

1. Avvia il server ,naviga nella directory **build** e avvia l'eseguibile del server. Questo terminale rimarrà attivo, in attesa di richieste dai client:
 - `cd build`
 - `./server`

Dovresti vedere un messaggio che indica che il server è avviato e in ascolto sulla sua FIFO principale

2. Avvia i client, Apri uno o più terminali separati. In ciascuno, naviga nella directory **build** ed esegui il client, fornendo il **percorso completo di un file** di cui desideri calcolare l'hash. Puoi eseguire più client contemporaneamente per testare la gestione della concorrenza del server:
 - `cd build`
 - `./client percorso/del/tuo/file.txt`
-